

UniVerse

A Campus Companion

Student Dashboard (Home)

The live home screen: greeting, the class happening now, a countdown to the next one, and today's list.

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Robi (student/teacher features)

Key files: features/dashboard/*

1. What it does

The student Home is a **read-only aggregation** of the routine - no new backend. It greets the student, shows a hero card for the class happening right now (or a countdown to the next class, or a 'done for today' message), a small stat strip, today's class list, quick actions, and a preview of recent alerts.

`StudentDashboardController` loads the cohort's routine once (`RoutineService.fetchForStudent`) and derives everything else in memory.

2. The three hero states (the key UX)

How the hero decides what to show

- 1 **Live class?** `liveClass` scans today's classes for one whose status is `live` (now between start and end). If found -> show the LiveClassCard.
- 2 **Otherwise, next class?** `nextClass` returns the first class today that hasn't started yet -> show NextClassCard with a countdown.
- 3 **Otherwise** -> show MessageHeroCard ('You're done for today').

This mirrors how a student actually thinks: 'what's on now? what's next? am I done?' - which makes the home screen instantly useful.

3. The derived values

getter	meaning
<code>todaysClasses</code>	Today's entries (by weekday), sorted by start time.
<code>liveClass</code>	The class running right now, if any.
<code>nextClass</code>	The first not-yet-started class today.
<code>remainingToday</code>	Count of classes today that haven't finished (live + upcoming).
<code>weeklyCount</code>	/ Totals for the stat strip.

todayCount

All of these are computed from the single loaded list against `DateTime.now()`. No extra queries, so the screen is fast and works offline once loaded.

4. How it stays in sync

The controller extends `SafeChangeNotifier` (a `ChangeNotifier` that won't call `notifyListeners()` after dispose). The screen wraps content in a `ListenableBuilder` listening to both the dashboard controller and the shared NotificationController, so the recent-alerts preview updates live.

5. Likely defense questions

question	answer
Does the dashboard hit a	No - it reuses RoutineService. It is pure derivation, zero new backend.
How is 'live' decided?	RoutineEntry.statusOn(now) -> getClassStatus compares now against the slot's start/end.
Why ChangeNotifier?	Built into Flutter, minimal boilerplate, perfect for our scale - the controller just calls notifyListeners() and the UI rebuilds.

What if there are no
classes today?

The hero gracefully shows the 'done for today' message card.